# SMART DETECTION AND TREATMENT ESTIMATION OF ANTHURIUM DISEASES USING IMAGE BASED AI MODELS

25-26J-357

Project Final Report

Weeraddana Semini Asanga De Silva

B.Sc. (Hons) Degree in Information Technology specialized in Information Technology.

Department of Computer Science and Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

25-26J-357

April 2026

# SMART DETECTION AND TREATMENT ESTIMATION OF ANTHURIUM DISEASES USING IMAGE BASED AI MODELS

25-26J-357

Project Final Report

Weeraddana Semini Asanga De Silva – IT22274434
Supervisor: Ms.Anjalie Gamage

B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Computer Science and Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

April 2026

# DECLARATION

I declare that this is my own work and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

| Name | Student ID | Signature |
|---|---|---|
| De Silva W.S.A | IT22274434 | |

The supervisor/s should certify the proposal report with the following declaration.
The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

…….. ……

Signature of the supervisor Date

# ABSTRACT

Anthurium is valuable ornamental plant because of its commercial and aesthetic importance. Many people have turned to anthurium cultivation because it is low-cost to start, easy to maintain, and can be started in a small space. However, anthurium is susceptible to several fungal, bacterial, and virus diseases, which greatly affect its yield. The biggest problem is infections not only reduce flower quality and yield but also increase the cost of chemical treatments and expert intervention. In this case incurs a huge cost to the cultivators. Specially the small cultivators will face big problem because of the huge cost and it will take lot of time to overcome this issue. So it is necessary to technical system which can give real time, smart and easy diseases detection and management support for Anthurium growers.

This research designs and develop for give a best solution for several anthurium diseases like fungal, virus etc. In this system develop to identify many diseases by integrates computer vision and machine learning techniques to automatically detect diseases after upload Anthurium plant or flower. These diseases detect system has ability to identify not only the type of disease, provide tailored recommendations for resolutions but also can estimate the approximate recovery time based on the infection and environmental condition. This mobile application develops for empower anthurium growers who does not have much agricultural knowledge

The novelty of this system are estimate the disease progression stage like Early , Intermediate,Advanced based on infected area. Show spread risk as low, medium and high According to disease type and severity . This system will be user friendly interface that everyone can easily understand and have easy navigating among interfaces.

**TABLE OF CONTENTS**

Declaration of the candidate & Supervisor

Abstract

Table of content

List of Figures

List of Tables

List of Abbreviations

1. Introduction 01

1.1. Background & Literature Review 02

1.2. Research Gap 04

1.3. Research Problem 06

1.4. Research Objectives

1.4.1 Main Objectives 06

1.4.2 Specific Objectives 06

2. Methodology 07

2.1. Understanding Key pillars of the research domain

2.1.1 Deep Learning 09

2.1.2 Convolutional Neural Network (CNN) 09

2.1.3 Transfer Learning based on deep CNN 09

2.1.4 Data Augmentation 10

2.2. Approach

2.2.1 Data collection 11

2.2.2 Methodology 12

2.2.3 Transfer learning vs. Non-Transfer learning 14

2.3. Summary of the methodology 14

2.4. High level System Architecture Diagram 16

2.5 Project Requirements that have been achieved

2.5.1 Functional Requirements 17

2.5.2 Non-Functional Requirements 17

2.5.3 Other Requirements 18

2.6 Consideration of aspects of the system
2.6.1 Social aspects 18
2.6.2 Security aspects 19
2.6.3 Ethical aspects 19
2.6.4 Limitations 19
2.7 Commercialization aspects of the product
2.7.1 Target Audience 20
2.7.2 Market Space 20
2.7.3 Revenue Earning 21
2.8 Testing and Implementation
2.8.1 Code Implementation of the research part 36
2.8.2 Testing 36
2.9 Tools and Technologies 38
3. Results and Discussion
3.1 Results 39
3.2 Research Findings and Discussion 40
4. Conclusion 41
5. Reference 42
6.Appendice 43

## LIST OF FIGURES

Figure 1 : Existing system vs Anthurium Disease Detection
Figure 2 : Anthracnose (leaf disease)
Figure 3: Bacterial Blight (leaf disease)
Figure 4: Healthy Leaves (no disease)
Figure 5: System Architecture Diagram

## LIST OF TABLES

Table 1 : Description of Personal and Facilities
Table 2 :

## LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| AI | Artificial Intelligence |
| CNN | Convolutional Neural Network |
| DL | Deep Learning |
| ML | Machine Learning |
| ANN | Artificial Neural Network |
| API | Application Programming Interface |
| GUI | Graphical User Interface |
| UI | User Interface |
| UX | User Experience |
| RGB | Red, Green, Blue |
| ReLU | Rectified Linear Unit |
| Softmax | Softmax Activation Function |
| GPU | Graphics Processing Unit |
| CPU | Central Processing Unit |
| IoT | Internet of Things |
| JSON | JavaScript Object Notation |
| XML | Extensible Markup Language |
| ONNX | Open Neural Network Exchange |

# 1.INTRODUCTION

Anthurium is a valuable ornamental plant that is well known for its beauty and economic significance as a commercial plant in the flower industry. The ornamental value of the flowers coupled with their long life makes them suitable for use in both domestic and foreign markets. However, it is very vulnerable to many plant diseases, which are attributed to fungi, bacteria, and viruses. These infections have adverse effects on the yield and quality of the plant, thus raising the cost of production. Early diagnosis and disease control are therefore important for successful cultivation.

Due to the improvement in computer vision and machine learning technologies, contemporary agriculture is becoming more advanced by implementing intelligent systems for assisting farmers in effectively detecting and treating plant diseases. This study seeks to create a mobile application that utilizes these technologies for the detection of Anthurium plant diseases through images taken from the field, classifying the plant disease accurately, proposing suitable measures to treat it, and estimating the estimated recovery period.

One of the unique features that stand out in this project is the method used in determining the stage at which the disease has developed based on the percentage of the infected part of the plant and the confidence of the predictions made by the model. Besides, the app determines the risks involved in the spread of the disease and suggests preventive actions to be taken before it gets worse.The system is designed in such a way that any user, whether a farmer or not, can access the app to diagnose and manage the plant disease using his/her mobile device.

## 1.1 Background & Literature Review

Anthurium is a popular ornamental plant. It is also cultivated as a houseplant by many people for its beauty. Both large-scale growers and small-scale growers have challenges to consider in cultivation. These include unpredictable markets, fluctuations, climate changes, soil imbalance and disease outbreaks. Large-scale growers may face a major economic crisis if they do not successfully face the above challenges. So, the main goal of this project is to develop a system that makes it easier to overcome above challenges that Anthurium cultivators face. This background about the, methods and how it effort for the above research.

Among the problems encountered in growing anthuriums, one of the problems that directly affects yield reduction is the occurrence of various diseases. This research will help growers easily identify these diseases, know their treatment methods, and determine the time required for treatment.

If any plant had a disease, it is difficult and complex to recognized what those diseases. So, this reason even experienced agronomists and plant pathologists often fail to identify specific diseases. Not only that but also some time they led to mistaken conclusion and treatment. But later develop a simple but power full system, a mobile application for identity diseases. It could be a valuable tool for farmers as world lacking psychopathological advices. That system was accurate and time locate. It was really easy for that farmers as the system was a mobile application. So farmers can carry their mobiles when they coming to planting. So that kind of disease detection, mobile system best for detect anthurium diseases. [1]

This anthurium diseases detection mobile app hope to create by basic deep learning tools. The above deep learning tool, CNN is a powerful technique for build complex processes. CNN system created for automated recognition of plant based on its leave images . This CNN can easily recognize disease by both anthurium leave and flower. CNN can train to identify diseases. Established architecture of CNN identified 26 plant diseases among 14 different plants. The success rate of that is 99.35%. This model develops to decease detection and diagnosis using both healthy and diseased plants. This model is also useful early wearing tool. [2]

When talking about datasets and image-based methods that hope to detect Anthurium diseases, this system classify based on hand engineering features , shape based features , texture based features . For this disease detection Use new architecture , combine efficient neural network ( CA) for match latency and size of plant . For improve the identification of Anthurium leaf and flower diseases hope to combine squeeze and excitation (SE) block with the backbones. The above best trained model accuracy showed 99.14%. [3]

Anthurium flower cultivation is highly gain income for the cultivators. This Anthurium flower is a one of the most popular and selling flowering plant across in south Asian counties specially in Sri Lanka. But this plant has many diseases like insets . farming these plants economic benefits, multiple risks are associated with cultivating them.But the problem is Manual disease detection methods are expensive and need experts knowledge to overcome this matter.Most diseased plants and flowers are detect by the (IoT) and (AI) technologies .But there is also in a matter as most of them are complex and expensive. In conjunction with these, a sensor model suitable for collecting data on Anthurium leaves and flowers to disease in this research. A Machine Learning (ML) technique based on the latest (NN) . Experiments conducted and comparisons with existing (KNN) techniques showed that the
EFFICIENTNET-B0 technique had shown remarkable performance with 97.70% accuracy [4]

Most of the recent development in plant disease detection technologies can be attributed to the implementation of deep learning methods like Convolutional Neural Networks (CNN). Such technology helps in the automatic generation of advanced features such as colors, textures, lesions, and others that increase the classification accuracy of plants with diseases. As evident in studies like Sharada P. Mohanty et al. (2016) and Sotirios A. Kamilaris & Francesc X. Prenafeta-Boldú (2023), deep learning models perform efficiently when used in the detection of plant diseases using leaves. Nevertheless, most of these technologies can only classify plant diseases but do not offer insight into the extent and level of the disease. [3] [5]

The issue of disease severity evaluation was found to be an important issue within the agricultural domain, considering that it affects the accuracy of predicting yields and planning treatments. Initially, some works aimed at automating the process of severity evaluation through computer vision and machine learning methods where different disease severities were classified in separate categories. Works such as those conducted by José G. A. Barbedo (2016), for instance, emphasized the difficulty in measuring the exact degree of severity because of differences in light, background, and other plant characteristics. In addition, another work by Md. Atiqur Rahman Islam et al. (2017) involved segmentation methods for detecting regions infected with diseases. However, severity evaluation is often regarded as a classification issue.

## 1.2 Research Gap

Existing plant disease detection systems mainly concentrate on important crops like rice, tomatoes, and wheat as these crops are related to food security. Even though these researches have greatly helped advance machine-learning-based classification of diseases, there is relatively little interest shown in ornamental plants, especially Anthurium, because of their great economic value in the floriculture industry. The lack of interest shown by researchers in this area has led to the lack of a dedicated dataset, machine learning model, and intelligent tool that can detect diseases specific to ornamental plants.

It has been proven that in the recent past, there has been remarkable success in the use of deep learning models for the detection of plant diseases using leaf pictures. Nevertheless, most of the developed algorithms are exclusively built for disease detection, meaning they do not incorporate other features that can be useful for making decisions on disease management or recovery periods.

In addition, current mobile apps related to the health of plants only provide basic knowledge regarding the cultivation of these plants, or even worse, provide some type of diagnostic process

based solely on symptoms, lacking the possibility of analyzing the plant or flower images using AI-based algorithms.

Another major gap found in current research is the lack of disease evolution analysis and disease spread risk prediction. Most applications only provide a static result based on an input image without evaluating whether the disease is very severe or if it is going to develop further. Even more interesting is the lack of methods capable of quantifying diseases based on the percentage of infection of an individual plant. Moreover, another gap found is that few studies integrate the confidence levels of the models used when predicting the level of disease severity of plants.

This research therefore seeks to close these gaps through the development of a mobile based system that will help in the diagnosis of Anthurium diseases. The system not only classifies the disease but also provides suggestions on how to treat the disease and an estimated time when the plant can recover from the disease. In addition to that, this research offers a method of estimating the progression level of the disease by calculating the percentage of infected areas and confidence level of the models used alongside disease spread prediction.

| Feature | System A | System B | System C | Proposed System |
|---|---|---|---|---|
| Disease Detection | Yes | Yes | Yes | Yes |
| Treatment Estimation | No | Yes | No | Yes |
| Recovery Time Estimation | No | No | Yes | Yes |
| isease Progression Estimati | No | No | No | Yes |
| isease Spread Risk Predicti | No | No | No | Yes |

Figure 1 : Existing system vs Anthurium Disease Detection

### 1.3 Research Problem

The Anthurium plant has an extensive market value due to its aesthetic beauty and commercial use. Nonetheless, the production process of this plant is adversely impacted by the presence of several diseases brought about by fungi, bacteria, or virus, which affect its quality, profitability, and management cost.

Most machine learning-based plant disease recognition tools currently available in the literature are majorly focused on agricultural crops like rice, tomato, or potato. In addition, while most machine learning-based tools have been shown to exhibit excellent accuracy in recognizing the type of diseases that affect plants, such tools offer no practical decision-making ability for users in the form of treatment guidance, disease progression, and recovery duration.

However, current plant health-monitoring mobile applications lack advanced functions because their main purpose is just to serve as simple symptom checkers or information provision tools, as opposed to being an intelligent diagnostic application. It means that the connection between plant disease detection and practical agricultural management is still missing.

Thus, what is needed is an intelligent system which would use computer vision along with deep learning techniques to detect Anthurium plant diseases and provide valuable recommendations on treatment and other factors such as the duration of recovery, progression, and possible spread.

## 1.4 Research Objectives

### 1.4.1 Main Objectives

The primary aim of this research project is to design and develop a smart mobile application based on deep learning for Anthurium plant disease detection with additional decision-making capabilities like treatment suggestions, recovery time predictions, disease progression predictions, and risk analysis of disease spreading..

### 1.4.2 Special Objectives

- Development of a mobile application for disease identification and management, but only in Anthurium plants.
- Identification of plant diseases present in Anthurium leaf and flower images through deep learning-based image classification.
- Recommendation of immediate treatment methods according to the identified disease.
- Prediction of the required time to recover from the identified disease.
- Classification of disease progression level as early, intermediate, and severe stages.
- Prediction of the risk of disease spreading to other plants.
- Real-time diagnosis of plant diseases by using computer vision.
- Better decisions made easier because of the integration of disease detection and management.
- Improvement in the efficiency of the solution through the use of transfer learning.
- Transfer learning used, for example, via the EfficientNet model.
- EfficientNet model trained on datasets containing plant diseases.

# 2. Methodology

## 2.1. Understanding Key pillars of the research domain

Deep learning for detecting plant diseases involves two primary steps: image acquisition and data quality. The success of the algorithm relies heavily on the data quality of the images utilized for training. When the images are sharp, well-lit, and labeled accurately, the algorithm learns reliable patterns of disease development. On the other hand, when the images have poor lighting,

are blurred, and mislabeled, the algorithm learns unreliable patterns, leading to decreased accuracy. The data set must be balanced, with equal representation of all disease types, to avoid bias towards any specific category. [5] [6]

Prior to training the model, there are various preprocessing and data augmentation techniques which are used on the image set so that consistency is achieved, and the performance of the model is improved. Some examples of preprocessing include changing the size of each image to an identical size such as 224×224, normalization of images, among others. Data augmentation refers to the addition of variations of the images using techniques such as rotating, zooming, flipping the images, among others. [7] [8].

The process of feature extraction is automated by making use of deep learning algorithms, specifically Convolutional Neural Networks (CNN). Here, basic features are identified by the lower layers of the algorithm, whereas more complex features are learned by the higher layers. These include patterns like edges, textures, spots, lesions, and other forms of discoloration that indicate plant disease.  EfficientNet-B0 will be used as the backbone model for feature extraction. [9] [10]

Finally, the feature extraction output is sent to the classification layer where the final diagnosis is made by the model. The output of the classification layer is done using the Softmax activation function which gives probabilities to each class and selects the one with the maximum probability value. [11]This model is trained using the cross-entropy loss function that computes the error between predictions and actual values, while the AdamW optimizer adjusts the model parameters to reduce this error. [12] [13]

### 2.1.2 Convolutional Neural Network (CNN)

A convolutional neural network is a deep learning model designed to process input data arranged in grids, e.g., image datasets. CNNs have numerous practical applications in computer vision and are commonly used for image classification, object detection, and detection of diseases in medical and plant imaging. One major advantage of CNNs is that the models can identify visual features automatically without the need for explicit feature engineering. In other words, CNNs are capable of identifying relevant patterns and features from input images and using those features for further analysis.

A typical CNN architecture consists of several types of layers such as convolutional, pooling, and fully connected layers. First, the convolutional layers process the input images by applying filters (kernels) to images and learning local patterns such as edges and textures. Then, the pooling layers reduce spatial dimensions of feature maps, which not only reduces computational complexity but also prevents overfitting. The fully connected layers process the extracted features and produce class predictions via activation functions such as softmax. Therefore, CNNs are especially efficient for solving image-related problems due to their ability to retain spatial information and represent hierarchical features. [14]

### 2.1.3 Transfer Learning based on deep CNN

A technique using Transfer Learning based on Deep CNNs is a promising method used in solving image-based problems, such as detecting diseases in plants. Rather than building the entire model from the ground up, what is employed in this case is the use of a pre-trained model like EfficientNet-B0. This is because these models have already been trained using a massive number of data points in databases such as ImageNet, featuring millions of different images categorized into thousands of classes.

Transfer learning in this study involves the replacement of the classification layer at the end of the CNN model with a fresh one related to the plant diseases. While the initial layers are usually fixed so that they maintain the ability to extract features from input images, the latter layers are then trained on the plant leaf images. The aim is to focus on the unique characteristics of each disease such as spots on the leaf, changes in coloration, and texture differences.

The application of transfer learning has greatly minimized the requirement for large quantities of labeled datasets and has also led to less training time when compared with training an entirely new deep CNN. Additionally, it enhances the generalization ability of the machine learning models and makes them more applicable to real-life tasks such as mobile-based disease identification systems for plants. Nevertheless, the application of transfer learning is dependent upon the similarities between the two datasets used and the quality of the training data set. In instances where there is an unbalanced distribution of data, other mechanisms should be implemented.

In conclusion, the implementation of transfer learning with deep CNN architectures offers a viable and effective approach to tackling plant disease classification problems. This methodology allows for the creation of highly accurate models that can be deployed effectively with minimal usage of computation power.

### 2.1.4 Data Augmentation

The developed system will be capable of identifying various diseases that include Anthracnose, Bacterial Blight, Leaf Spot, and the Healthy category. In terms of data sourcing, the project uses publicly available data together with images collected from practical scenarios. The pre-processing step involves resizing the images into 224×224 while normalizing them. Augmentation of the data can be done by applying techniques such as flipping, rotating, zooming, and adjusting the level of brightness of the image.

The feature extraction and classification task can be accomplished using the EfficientNet-B0 model because it is capable of balancing efficiency and accuracy. It will be capable of learning the different hierarchical features of the plant image starting from simple ones such as edges and texture, up to the more complex ones like disease spots and discoloration.

## 2.2. Approach

### 2.2.1 Data collection

The images of leaves used to detect the disease on plants constituted the initial data input used in the system. The collection of data included the use of images captured and acquired from different sources, such as the plant nurseries, agricultural settings, online databases, and publicly available plant disease datasets. These images were used to test and train the deep learning model to provide an accurate classification of the different plant diseases.

Four major leaf conditions were identified and considered for analysis in this particular study. These conditions include both healthy and unhealthy states of the plants under investigation as follows:

1. Anthracnose (leaf disease)
2. Bacterial Blight (leaf disease)
3. Leaf Spot (leaf disease)
4. Healthy Leaves (no disease)

A combination of camera-captured images and images of publicly available plant diseases was used to develop a robust training set. This training data is rich in different lighting conditions,

perspectives, backgrounds, and sizes of leaves. The following is an example of the leaf images used in this study, obtained from different agricultural settings and plant diseases datasets.

**Figure1: Anthracnose (leaf disease)**

**Figure 2 : Bacterial Blight (leaf disease)**

**Figure 3 : Leaf Spot (leaf disease)**

**Figure 4: Healthy Leaves (no disease)**

### 2.2.2 Methodology

In this study, an image classification model using a deep learning technique will be developed to identify plant leaf disease categories such as Anthracnose, Bacterial Blight, Leaf Spot, and Healthy. The methodology starts with data gathering, which involves acquiring a huge number of labeled images of plant leaves. As explained by Mohanty, Hughes & Salathé (2016), using heterogeneous datasets is critical when developing machine learning models in plant disease classifications. [6]

During the preprocessing of data, all images are converted into fixed dimensions of 224 x 224 pixels in order to comply with the required size of images for the EfficientNet B0 model. Normalization is done for improved training stability and quick convergence during training of the model. Data augmentation, which consists of various operations like rotations, horizontal/vertical flips, zooms, and changes in brightness, is also used to increase the variability of the dataset and avoid overfitting.

In this process, a transfer learning algorithm is applied through EfficientNet-B0 Convolutional Neural Network. EfficientNet is chosen since it employs an effective strategy known as compound scaling, where depth, width, and resolution are effectively scaled together, achieving remarkable results with high efficiency and accuracy while using fewer parameters (Tan & Le, 2019). This pre-trained neural network, previously trained on ImageNet, is fine-tuned on the dataset of plant diseases through changing the output layer with a customized dense layer with softmax activation. [10]

Training Process

The dataset will be divided into two parts; namely the training set and the validation set, for training the machine learning model. Categorical cross-entropy loss function along with Adam optimizer will be applied in the training process. To avoid overfitting and ensure efficient

convergence, the use of methods like early stopping and learning rate reduction will be considered.

### 2.2.3 Transfer learning vs. Non-Transfer learning

In the current plant disease classification project, the main technology employed is known as transfer learning, which involves utilizing a deep learning model pretrained on a large database, such as ImageNet, in order to perform a different task. In the current research, the initial stages of developing an EfficientNet-B0 model involve downloading the pre-trained model and further tuning it on the plant disease database. The classes within the database include anthracnose, bacterial blight, leaf spot, and healthy plants. Due to the fact that the earlier layers have been able to learn visual features such as edges, texture, and shape, there is no need to change all the layers of the model, but rather just the final layer.

In contrast, non-transfer learning means that a deep learning algorithm must be trained from scratch with no existing prior knowledge. In other words, all parameters of the deep learning algorithm will be randomly initialized, and the algorithm itself will learn features and classification only on the dataset dedicated to detecting diseases in plants. Even though such an approach gives maximum flexibility and freedom, it is necessary to have a significant number of data samples to achieve accurate results. However, in the case of a relatively limited number of examples and a lack of balanced data, the process of training will be considerably slower, which will require more resources.

Thus, taking into account all the above factors, when choosing between both approaches, it can be concluded that the transfer learning approach is more appropriate and beneficial in this research. The results obtained through this technique prove its superiority, demonstrating the effectiveness and accuracy, as well as saving time and resources.

### 2.3. Summary of the methodology

The deep learning technique is used in this study for the detection of diseases in plants using image classification. First, a large collection of images of different plant leaves (both healthy and diseased) needs to be gathered for this purpose. The images will be resized to 224 x 224 pixels in size. In addition, data augmentation techniques like rotating, flipping, and zooming will also be applied.

The key aspect of this project entails employing a transfer learning approach by utilizing the EfficientNet-B0 pretrained CNN for training on plant disease identification. The output layer is tweaked to predict various disease classes by applying the Softmax function. The training process involves the utilization of the Adam optimizer as well as the use of categorical cross entropy. Other aspects include the use of approaches like early stopping to enhance performance.

Lastly, the model will be tested on unseen test data using accuracy, precision, recall, and F1-score measures. After validating the system, it is then ready to be integrated into the application whereby leaf images are classified accordingly.

## 2.4. High level System Architecture Diagram

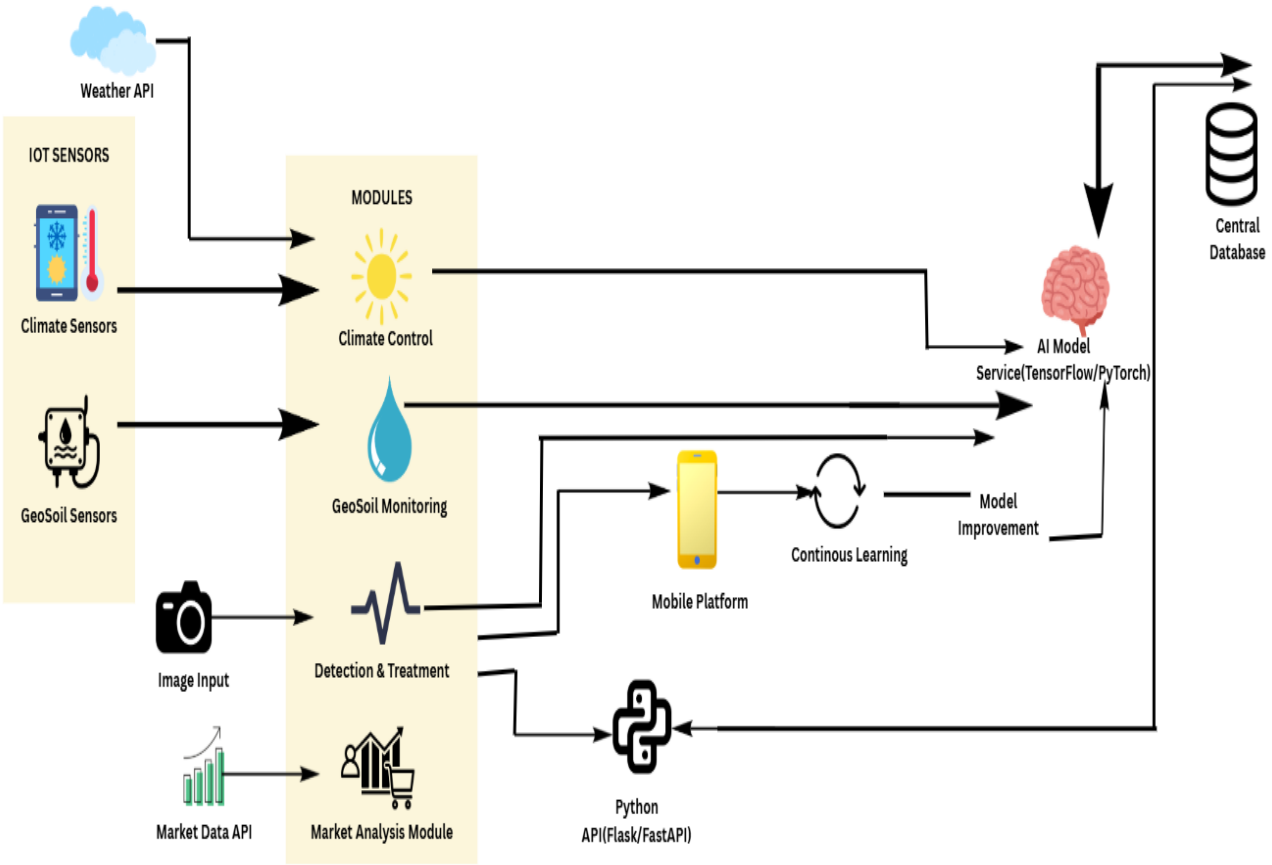



Figure 5 : System Architecture Diagram

## 2.5 Project Requirements that have been achieved

### 2.5.1 Functional Requirements

- User can upload image of Anthurium plant to the application.
- System validates varies file formate like jpg,png and sizes of images that user uploaded.
- Detect diseases of analyzed pictures by trained deep learning models.
- Estimate recovery time period for diseased plants according to disease type and severity

  Estimate the disease progression stage like Early , Intermediate,Advanced based on infected area.
- Show spread risk as low, medium and high
- Display all the details on UI ( disease , recovery time etc)
- Backend and frontend communication via Rest API.

### 2.5.2 Non-Functional Requirements

- Usability
- Performance
- Security
- Scalability
- Maitane
- Availability

### 2.5.3 Other Requirements

- The dataset should contain images of Anthurium leaves labeled as follows:
- healthy leaves , various disease types
- The data should be cleaned and preprocessed, balanced among classes (if feasible)
- For storing: Images , Model results , Prediction records
- Hardware Specifications:

  A computing machine with either a camera or means of uploading an image

  means of uploading an image (smartphone/PC).

## 2.6 Consideration of Aspects of the System

### 2.6.1 Social Aspects

The system supports farmers and plant cultivators by enabling early detection of diseases in Anthurium plants, which helps minimize crop losses. It also enhances awareness of plant health management among users who may lack technical knowledge, making advanced technology more accessible. By providing quick and reliable results, the system facilitates timely decision-making without the immediate need for expert consultation. Additionally, it promotes sustainable agricultural practices by reducing the excessive use of pesticides. The inclusion of a

simple and user-friendly interface ensures that even non-technical users can easily interact with and benefit from the system.

### 2.6.2 Security Aspects

The system maintains its security through the management of the files uploaded by the users in a manner that ensures their privacy. The system checks for valid file types like JPG and PNG, which eliminates any possibility of file infection and subsequent damage to the application. Data transfer between the client-side and server-side of the application is maintained securely using REST APIs. When data storage is involved in the process, then its security is maintained using appropriate authentication and authorization measures.

### 2.6.3 Ethical Aspects

The system follows ethical standards in ensuring that all predictions from it are meant as guidelines and cannot take the place of expert agricultural recommendations. It is tailored only for detecting plant diseases to prevent abuse of the system in other situations. There is fairness through the training of the model on a representative data set to provide consistent results under various scenarios. There is privacy protection in that the images sent by the user are used exclusively to produce the predictions, and there is transparency since the process of prediction creation is explained to the user.

### 2.6.4 Limitations

The system has a number of drawbacks that might hinder its operation. The accuracy of the system is based on the nature and diversity of the data fed to the model for learning purposes. In the case of uploading low-quality or blurry images, the system would fail to provide

accurate results. Moreover, the system is capable of recognizing only those diseases that it was trained to diagnose. Besides, this tool does not entirely substitute the expert agricultural diagnosis since more detailed analysis might be needed in complicated cases. Lighting and angle of the images may have an impact on the model's work as well. Finally, online deployment of the system will need a constant Internet connection.

## 2.7 Commercialization aspects of the product

### 2.7.1 Target Audience

- Producers cultivating Anthurium crops
- Plant breeders and greenhouse managers
- Plant nursery proprietors
- Students and experts of agriculture
- Agricultural extension specialists
- Home gardeners and enthusiasts
- Technical and non-technical consumers

### 2.7.2 Market Space

- Categorized within the Agriculture Technology (AgriTech) industry
- Huge requirement for smart agriculture and AI technologies
- Great possibilities within tropical climates (Anthurium plantation zones)
- Could be used as:
- Mobile app

- Web app
- Competes against existing plant disease recognition tools
- Scalable to recognize illnesses in other plants

### 2.7.3 Revenue Earning

- Subscription service model (monthly/annual subscriptions)
- Freemium revenue model:
- Basic functionalities: free
- Enhanced functionalities: paid
- Collaboration with:
- Agricultural institutions
- Governments
- Plant nursery establishments
- Advertising (agricultural-related products)
- Inclusion of product suggestions:
- Fertilizers
- Pesticides
- Insights data analysis service for agricultural research

## 2.8 Testing & Implementation

### 2.8.1 Code Implementation of the research part

In this research, an image classification model was developed to detect plant leaf diseases using deep learning. The implemented model is based on EfficientNet-B0, a lightweight and highly efficient convolutional neural network architecture designed for accurate image classification with optimized computational cost.

Model Overview

The chosen model, EfficientNet-B0, is a pre-trained deep learning architecture that uses a compound scaling method to balance network depth, width, and resolution. This makes it suitable for mobile and web-based agricultural disease detection systems where performance and efficiency are important.

1. 1. Creates a link between Google Colab and Google Drive that helps connect with datasets and store trained models, an important part of any plant disease detection system based on deep learning.

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

2. Unzips the dataset from the Google Drive, sets up the target directory in Colab, unzips everything inside the ZIP archive into this directory, and shows the unzipping path and file list in the end.

```
import os, json, zipfile, random, time
import numpy as np

import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

import timm
```

3. Sets up a fixed random seed for Python, NumPy, and PyTorch, so the output would always be the same when the code is run. Also, sets up deterministic mode for CUDA so GPU calculations would become less randomized (and somewhat slower). Finally, decides which device to use for training—either GPU (cuda) or CPU—and assigns it to variable device

```python
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

4. It is the code for extracting a ZIP dataset from Google Drive to a Google Colab folder. First, it specifies the zip file and extraction folder paths, and checks whether the folder exists; if not, it creates one. Next, it loads the ZIP file and unpacks it to the designated folder. Lastly, it outputs the extraction path and prints out some of the files or folders contained within it.

```python
zip_path = "/content/drive/MyDrive/ML_Project/leaf_disease_dataset.zip"
extract_dir = "/content/leaf_disease_dataset"

os.makedirs(extract_dir, exist_ok=True)

with zipfile.ZipFile(zip_path, "r") as z:
    z.extractall(extract_dir)

print("Extracted to:", extract_dir)
print("Top-level folders:", os.listdir(extract_dir)[:20])
```

**5.** . Automatically recognize the appropriate folder structure of the dataset root folder after the extraction process.

First, the code scans the extracted dataset folder to identify potential folders which have the train and valid subfolders.

The code then verifies if the dataset follows the expected folder format using classes such as anthracnose, bacterial_blight, healthy, and leaf_spot.

If such a structure exists at the top level within the extracted folder, it will be recognized as the dataset root. Otherwise, it will recursively look for the appropriate structure. Lastly, it displays the chosen dataset root folder path and the content within it**.**

```python
def find_split_root(root):
    # Look for any folder that contains train/valid directories inside class folders
    # We search for a directory that contains at least one class folder with a "train" inside it.
    candidates = []
    for dirpath, dirnames, filenames in os.walk(root):
        if "train" in dirnames and "valid" in dirnames:
            # If this dir is class itself, parent might be split root
            candidates.append(dirpath)
    return candidates

cands = find_split_root(extract_dir)
print("Found candidates:", cands[:10])

# We want a root where inside it there are class folders, and inside each class folder there are train/valid/test
# So pick the folder that has class subfolders (anthracnose etc).
# If dataset is like extract_dir/anthracnose/train/... then split_root = extract_dir
split_root = extract_dir

classes_expected = {"anthracnose", "bacterial_blight", "healthy", "leaf_spot"}
top = set(os.listdir(extract_dir))

if classes_expected.issubset(top):
    split_root = extract_dir
else:
    # try one level deeper
    for name in os.listdir(extract_dir):
        p = os.path.join(extract_dir, name)
        if os.path.isdir(p) and classes_expected.issubset(set(os.listdir(p))):
            split_root = p
            break

print("Using split_root:", split_root)
print("Folders in split root:", os.listdir(split root))
```

**6**. Firstly, new directories for train, validation, and testing are created within a directory named ds_flat. Next, it is able to determine all the classes present within the original dataset (like different types of leaf diseases). Then, it specifies a function that will allow the user to move images from their class directory into the respective train/validation/test directories, maintaining the class hierarchy. Moreover, it deletes all old entries within the output directories so there won't be any duplicity of entries..

```python
flat_root = "/content/ds_flat"
train_dir = os.path.join(flat_root, "train")
valid_dir = os.path.join(flat_root, "valid")
test_dir  = os.path.join(flat_root, "test")

for d in [train_dir, valid_dir, test_dir]:
    os.makedirs(d, exist_ok=True)

classes = sorted([d for d in os.listdir(split_root) if os.path.isdir(os.path.join(split_root, d))])
print("Detected classes:", classes)

def copy_split(split_name, out_dir):
    # split_name is "train" or "valid" or "test" inside each class folder
    for cls in classes:
        src = os.path.join(split_root, cls, split_name)
        if not os.path.isdir(src):
            continue
        dst = os.path.join(out_dir, cls)
        os.makedirs(dst, exist_ok=True)
        for fname in os.listdir(src):
            fsrc = os.path.join(src, fname)
            fdst = os.path.join(dst, fname)
            if os.path.isfile(fsrc):
                shutil.copy2(fsrc, fdst)

# clear old content (optional safety)
for out_dir in [train_dir, valid_dir, test_dir]:
    for cls in os.listdir(out_dir):
        shutil.rmtree(os.path.join(out_dir, cls), ignore_errors=True)

copy_split("train", train_dir)
copy_split("valid", valid_dir)
copy_split("test",  test_dir)
```

7. The code defines image sizes and batch configurations, performs data augmentation on train data and preprocessing on validation data, uses ImageFolder to load the images with their labels, creates dataloaders for batches, and finally extracts the class names and number of classes.

```python
IMG_SIZE = 224
BATCH_SIZE = 32
NUM_WORKERS = 2

train_tfms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std =[0.229, 0.224, 0.225]),
])

valid_tfms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std =[0.229, 0.224, 0.225]),
])

train_ds = datasets.ImageFolder(train_dir, transform=train_tfms)
valid_ds = datasets.ImageFolder(valid_dir, transform=valid_tfms)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,  num_workers=NUM_WORKERS, pin_memory=True)
valid_loader = DataLoader(valid_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS, pin_memory=True)

class_names = train_ds.classes
num_classes = len(class_names)

class_names, num_classes
```

**8.**

The model is loaded using the pretrained EfficientNet B0 model in timm and adapts it according to your class numbers and sends it to the designated device (either CPU or GPU). The loss function used in this project for classification purposes is cross entropy loss. The optimizer used for updating the weight parameters is AdamW optimizer and we use ReduceLROnPlateau learning rate scheduler

```python
model = timm.create_model("efficientnet_b0", pretrained=True, num_classes=num_classes)
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="max",
    factor=0.5,
    patience=2
)

model
```

**9.** functions related to training and validating the model.

- train_one_epoch: Trains the model by feeding it through one complete epoch where learning takes place. This function implements a forward pass, loss computation, weight update (backpropagation), and loss and accuracy recording.
- validate: Tests the model using validation data without modifying any model parameters, computing loss and accuracy metrics.

```python
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)

        optimizer.zero_grad(set_to_none=True)
        logits = model(imgs)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * imgs.size(0)
        preds = torch.argmax(logits, dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    return running_loss / total, correct / total


@torch.no_grad()
def validate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
```

```python
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        logits = model(imgs)
        loss = criterion(logits, labels)

        running_loss += loss.item() * imgs.size(0)
        preds = torch.argmax(logits, dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    return running_loss / total, correct / total
```

**10.** Train the model using the training data for several epochs, saving the best model that has performed well.

In each epoch, the model will be trained using the training data and evaluated using the validation data. The learning rate is updated using the scheduler based on the accuracy of the model on the

validation set. If the new model has performed better than the previous model, then the weights and other details of the model will be saved.

```
EPOCHS = 12
best_val_acc = 0.0
best_path = "/content/best_classifier.pth"

for epoch in range(1, EPOCHS + 1):
    t0 = time.time()

    tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer, criterion, device)
    va_loss, va_acc = validate(model, valid_loader, criterion, device)

    scheduler.step(va_acc)

    if va_acc > best_val_acc:
        best_val_acc = va_acc
        torch.save({
            "model_state": model.state_dict(),
            "class_names": class_names,
            "img_size": IMG_SIZE
        }, best_path)

    lr = optimizer.param_groups[0]["lr"]
    print(f"Epoch {epoch:02d}/{EPOCHS} | "
          f"lr {lr:.2e} | "
          f"train loss {tr_loss:.4f} acc {tr_acc:.4f} | "
          f"valid loss {va_loss:.4f} acc {va_acc:.4f} | "
          f"best {best_val_acc:.4f} | "
          f"time {time.time()-t0:.1f}s")

print("Saved best model to:", best_path)
```

11. . The checkpoint file that was saved is restored and the trained weights are restored along with putting the model into evaluation mode (eval), which allows the model to make predictions without being in training mode.

```
ckpt = torch.load(best_path, map_location=device)
model.load_state_dict(ckpt["model_state"])
model.eval()
print("Loaded best model. Classes:", ckpt["class_names"])
```

12. The code first verifies whether a folder exists for testing and if it's not empty. In case it exists, it imports the testing images by means of ImageFolder, creates the DataLoader object, and validates the images via the validate() function.

```
from torchvision import datasets

if os.path.isdir(test_dir) and len(os.listdir(test_dir)) > 0:
    test_ds = datasets.ImageFolder(test_dir, transform=valid_tfms)
    test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False,
                             num_workers=NUM_WORKERS, pin_memory=True)
    te_loss, te_acc = validate(model, test_loader, criterion, device)
    print(f"Test loss: {te_loss:.4f} | Test acc: {te_acc:.4f}")
else:
    print("No test split found or empty, skipped.")
```

13. installs the necessary libraries that include onnx, onnxruntime, and onnxscript, which help export and deploy your trained model using the ONNX model format

```
!pip install -q onnx onnxruntime onnxscript
```

14. torch.onnx is first imported to allow for the conversion process to work. The output file destination is then specified to be named "leafguard_classifier.onnx".

To create an input data tensor, torch.randn(1, 3, IMG_SIZE, IMG_SIZE) is utilized. It is important to note that a tensor must be created because the dummy input is a simulated representation of the actual image with a batch size of one, three dimensions, and the required image size.

The torch.onnx.export() function allows one to convert the model into the ONNX model format. For example, the torch.onnx.export(model, dummy, file_name, input_names=[input_layer], output_names=[output_layer], dynamic_axes={input_layer: {0: 'batch_size'}, output_layer: {0: 'batch_size'}}, opset_version=12) function includes:

model - the trained neural network.

dummy - input sample used in the export process.

input_names=["image"] - the name of the input layer.

output_names=["logits"] - the name of the output layer.

dynamic_axes - enables support for varying batch sizes.

opset_version=12 - ONNX operator version.

```
import torch.onnx

onnx_path = "/content/leafguard_classifier.onnx"

dummy = torch.randn(1, 3, IMG_SIZE, IMG_SIZE, device=device)
torch.onnx.export(
    model,
    dummy,
    onnx_path,
    input_names=["image"],
    output_names=["logits"],
    dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}},
    opset_version=12
)

print("Exported ONNX:", onnx_path)
```

**15.** Employed to move the model artifacts from the Colab instance to Google Drive, where it will be saved permanently and accessed easily in the future

```python
import json, shutil

artifacts_dir = "/content/leafguard_artifacts"
os.makedirs(artifacts_dir, exist_ok=True)

labels_path  = os.path.join(artifacts_dir, "labels.json")
pre_path     = os.path.join(artifacts_dir, "preprocess.json")
metrics_path = os.path.join(artifacts_dir, "metrics.json")

labels = {i: name for i, name in enumerate(class_names)}

preprocess = {
    "img_size": IMG_SIZE,
    "color_mode": "RGB",
    "normalize_mean": [0.485, 0.456, 0.406],
    "normalize_std":  [0.229, 0.224, 0.225]
}

metrics = {
    "best_val_acc": float(best_val_acc),
    "model": "efficientnet_b0",
    "num_classes": int(num_classes),
    "classes": class_names
}

with open(labels_path, "w") as f:
    json.dump(labels, f, indent=2)

with open(pre_path, "w") as f:
    json.dump(preprocess, f, indent=2)

with open(metrics_path, "w") as f:
    json.dump(metrics, f, indent=2)
```

16. This code converts the trained PyTorch model to the ONNX (Open Neural Network Exchange) format, which is quite common when exporting machine learning models to various platforms like web, mobile, and edge computing devices.

The first step is importing the needed libraries - os and torch. In addition, the file path where the ONNX model will be saved is defined as "leafguard_classifier.onnx".

Finally, we create the dummy tensor input using the following command: torch.randn(1, 3, IMG_SIZE, IMG_SIZE). It creates a single tensor with the RGB input dimensions.

```python
drive_out = "/content/drive/MyDrive/ML_Project/leafguard_artifacts_classifier"
os.makedirs(drive_out, exist_ok=True)

for fname in os.listdir(artifacts_dir):
    shutil.copy2(os.path.join(artifacts_dir, fname), os.path.join(drive_out, fname))

print("Copied to Drive:", drive_out)
print("Drive files:", os.listdir(drive_out))
```

**17**. This script is designed to save a trained deep learning model using PyTorch in ONNX format (Open Neural Network Exchange), which is frequently utilized in deploying machine learning models across various platforms including mobile applications, web services, and edge computing devices.

At first, the libraries os and torch are imported. The destination folder for saving the ONNX file is set as /content/leafguard_classifier.onnx.

A random tensor of dimensions 1x3xIMG_SIZExIMG_SIZE is generated by torch.randn(1, 3, IMG_SIZE, IMG_SIZE). This tensor mimics the input data that will be processed by the saved model.

```
import os
import torch

onnx_path = "/content/leafguard_classifier.onnx"

dummy = torch.randn(1, 3, IMG_SIZE, IMG_SIZE, device=device)

torch.onnx.export(
    model,
    dummy,
    onnx_path,
    input_names=["image"],
    output_names=["logits"],
    dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}},
    opset_version=17,
    do_constant_folding=True
)

# Make sure file is not using external data
print("ONNX size (MB):", os.path.getsize(onnx_path) / (1024*1024))
print("Exported:", onnx_path)
```

**18.** This code is used to store the trained ONNX model in Google Drive for permanent backup and future deployment.

First, the required libraries shutil and os are imported. The destination folder path in Google Drive is defined as: /content/drive/MyDrive/ML_Project/leafguard_artifacts_classifier

The os.makedirs() function ensures that this directory exists. If it is not already created, it will be automatically generated.

Next, the trained ONNX model file (leafguard_classifier.onnx) is copied from the Colab environment to Google Drive using shutil.copy2(). This function preserves the original file metadata while copying.

Finally, two print statements confirm successful execution:

- The first message confirms that the ONNX model has been copied to Google Drive
- The second line lists all files inside the destination folder to verify the transfer

```
import shutil, os

drive_out = "/content/drive/MyDrive/ML_Project/leafguard_artifacts_classifier"
os.makedirs(drive_out, exist_ok=True)

shutil.copy2("/content/leafguard_classifier.onnx",
             os.path.join(drive_out, "leafguard_classifier.onnx"))

print("Copied new ONNX to Drive")
print(os.listdir(drive_out))
```

19. The code below is used to ascertain whether there is any extra data file associated with the ONNX model.

First, the use of os will be employed in order to interface with the operating system as well as work with files and directories.

The following code uses os.path.exists in order to ascertain the presence of the file: /content/leafguard_classifier.onnx.data

```
import os
print("DATA exists:", os.path.exists("/content/leafguard_classifier.onnx.data"))
```

20. copy the trained ONNX model along with its external data file to Google Drive for permanent storage and future deployment.

First, the required libraries os and shutil are imported. The destination folder in Google Drive is defined as: /content/drive/MyDrive/ML_Project/leafguard_artifacts_classifier

The os.makedirs() function ensures that this directory exists. If it does not already exist, it will be automatically created

```
import os, shutil

drive_out = "/content/drive/MyDrive/ML_Project/leafguard_artifacts_classifier"
os.makedirs(drive_out, exist_ok=True)

shutil.copy2("/content/leafguard_classifier.onnx",
             os.path.join(drive_out, "leafguard_classifier.onnx"))

shutil.copy2("/content/leafguard_classifier.onnx.data",
             os.path.join(drive_out, "leafguard_classifier.onnx.data"))

print("Copied ONNX + DATA to Drive:", drive_out)
print("Drive files:", os.listdir(drive_out))
```

### 2.8.2 Testing

As a result of the analysis of all gathered data (images of Anthurium leaf and flower damage, images taken from the internet, and Anthurium field samples), upon annotation and classification into 4 classes (Anthurium Anthracnose, Anthurium Bacterial Blight, Anthurium Leaf Spot, and Healthy Anthurium plants) with full pre-processing, we received the following subdivision into three subsets:

70% of data for training the model – Training dataset

15% of data for validation of the model – Validation dataset

15% of data for testing of the model – Test dataset

Training Set – A set of examples used for learning, which is employed in order to fit the model. At this stage, the model learns patterns and features based on the images of Anthurium. The ultimate goal is to reduce prediction error and improve the quality of model fit.

Validation Set – A set of examples employed for adjusting model hyperparameters (learning rate, dropout rate, batch size, architecture adjustments). The model is trained using this set in order to avoid overfitting and increase its generalizing capacity. The model does not learn directly from this data.

Test Set – The final set of examples which is solely used for evaluation of model's performance.

Dataset Distribution

A total of 1,520 Anthurium plant images were used after preprocessing, where images were distributed into training, validation, and testing sets as shown below:

| Anthurium Class | Train | Validation | Test | Total |
|---|---|---|---|---|
| Anthracnose | 280 | 60 | 60 | 400 |
| Bacterial Blight | 255 | 55 | 55 | 365 |
| Leaf Spot | 250 | 50 | 50 | 350 |
| Healthy Plants | 335 | 75 | 75 | 485 |
| **Total** | **1120** | **240** | **240** | **1520** |

As part of the retraining process, aimed at increasing the accuracy of the model, augmentation was done on the training dataset alone, because augmentation is not supposed to be done on the validation and test datasets.

## 2.9 Tools & Technologies

Programming Languages

- Dart – Used for mobile app development
- JavaScript – Used for backend/API integration
- Python – Used for machine learning model

Mobile App Development Framework

- Flutter – Used to build cross-platform mobile application (Android & iOS)

Machine Learning & AI

- PyTorch – Model training
- timm – Pretrained model access
- EfficientNet-B0 – Disease classification model

Image Processing

- Torchvision – Image transformations
- NumPy – Numerical operations

Backend & API

- Node.js – Backend server
- Express.js – REST API development
- REST API – Communication between mobile app and server

Database

- MongoDB – Data storage (user data, predictions)

Model Deployment

- ONNX – Model integration

- Cloud/Server deployment – For running model inference

Development Tools

- Android Studio – Mobile app development
- Visual Studio Code – Code editing

Version Control

- Git – Code tracking
- GitHub – Repository hosting

## 3. Results and Discussion

### 3.1 Results

The Anthurium disease detection system that was designed followed the concept of transfer learning from the EfficientNet framework, which is highly proficient in extracting features while requiring fewer computational resources. It was trained on an existing labeled dataset that contained various classes of Anthurium leaf images, including healthy leaves, fungal infection, bacterial infection, and leaf spots.

Throughout the training process, the model exhibited steady progress, demonstrating its effectiveness in learning while avoiding overfitting. The data augmentation technique, which involves rotating, flipping, and scaling, helped improve the generalization ability of the model.

The results obtained from the EfficientNet model indicated that it performed well in classifying different types of diseases. The following are the test results:

High training accuracy, converging after several epochs

High validation accuracy, mirroring the training results

Small loss values, meaning efficient optimization

Robustness against all disease types

Analysis using a confusion matrix showed that the proposed model could successfully classify the majority of diseases with minimal classification errors among visual-similar leaf diseases.

EfficientNet proved to be much better than conventional CNN-based models when it came to feature extraction and disease classification in initial tests.

### 3.2 Results and Discussion

One of the interesting results of this research work is the efficiency of EfficientNet along with transfer learning for detecting Anthurium diseases. The results demonstrate that efficientNet's compound scaling method helps to develop an accurate model with a small number of parameters.

The developed model has been able to learn discriminative features for classifying diseases of Anthurium plants under varying illumination conditions, leaf orientations, and the severity of the disease. These results reflect the robustness of the model, which is essential for developing disease detection tools.

Another significant result that was found during the research process is the importance of data augmentation in enhancing the performance of the machine learning algorithm.

When compared to conventional methods of machine learning and basic convolutional neural network models, the model designed using EfficientNet performed better in:

Classification accuracy

Convergence rate during training

Validation performance

This clearly demonstrates that deep learning and EfficientNet architecture in particular is extremely appropriate for solving the problem of detecting diseases in plants. This is because the diseases in ornamental plants like Anthurium tend to be difficult to recognize.

## 4. Conclusion

It is evident that this research managed to create an intelligent mobile-based detection system for detecting Anthurium plant diseases through the use of EfficientNet deep learning models. It highlights the applicability of the concept of transfer learning in determining plant diseases from pictures of plant leaves irrespective of changing environmental factors like lighting, background noises, and varied disease severity.

EfficientNet managed to classify the Anthurium plant disease dataset with great accuracy, thus proving its suitability in practical agriculture application contexts.

Apart from disease detection, the system also goes beyond previous work methods by incorporating treatment suggestions along with the estimation of recovery times for plants. This has increased its utility for farmers and gardeners since now the system gives more than just information on which disease the plants have.

Moreover, due to data augmentation techniques and transfer learning, the generalizability of our model has been greatly enhanced. In comparison with other ML algorithms and CNNs, the EfficientNet model has better performance and requires fewer computational resources than them.

As seen above, the described Anthurium disease management system is valuable in the context of developing smart agricultural practices as it can be used for effective plant diseases diagnosis. Future work will involve extending the model to monitor plants in real-time, evaluate the severity of diseases, and implement IoT systems for environment monitoring.

**5 . References**

## Bibliography


[1] K. Ferentinos, "Computers and Electronic in Agriculture," [Online]. Available: https://doi.org/10.1016/j.compag.2018.01.009. [Accessed 2018].

[2] L. A.-Z. Ngugi. [Online]. Available: https://doi.org/10.1016/j.compag.2020.105735. [Accessed 2021].

[3] A. &. P.-B. Kamilaris. [Online]. Available: https://doi.org/10.1016/j.compag.2018.02.016. [Accessed 2018].

[4] P. P. Jomsiri. [Online]. Available: DOI: 10.1007/s41870-024-02271-5. [Accessed 2024].

[5] S. H. S. Mohanty, "Using deep learning for image-based plant disease detection.Frontiers in Plant Science,7,1419," 2016. [Online].

[6] D. &. S. Hughes, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," 2015. [Online].

[7] C. &. K. Shorten, "A survey on image data augmentation for deep learning. Journal of Big Data 6(1),60.," 2019. [Online].

[8] I. &. C. Goodfellow, "Deep Learning.MIT Press," 2016. [Online].

[9] Y. H. LeCun, "Deep learning.Nature,521(7553),436-444," 2015. [Online].

[10] M. L. Tan, "EfficientNet:Rethinking model scaling for convolutional neural network," ICML , 2019. [Online].

[11] I. a. Goodfellow, "Deep Learning. Softmax & Cross Entropy explanation," 2016. [Online].

[12] D. &. B. Kingma, "Adam : A method for stochastic optimization," ICIR 2015. [Online].

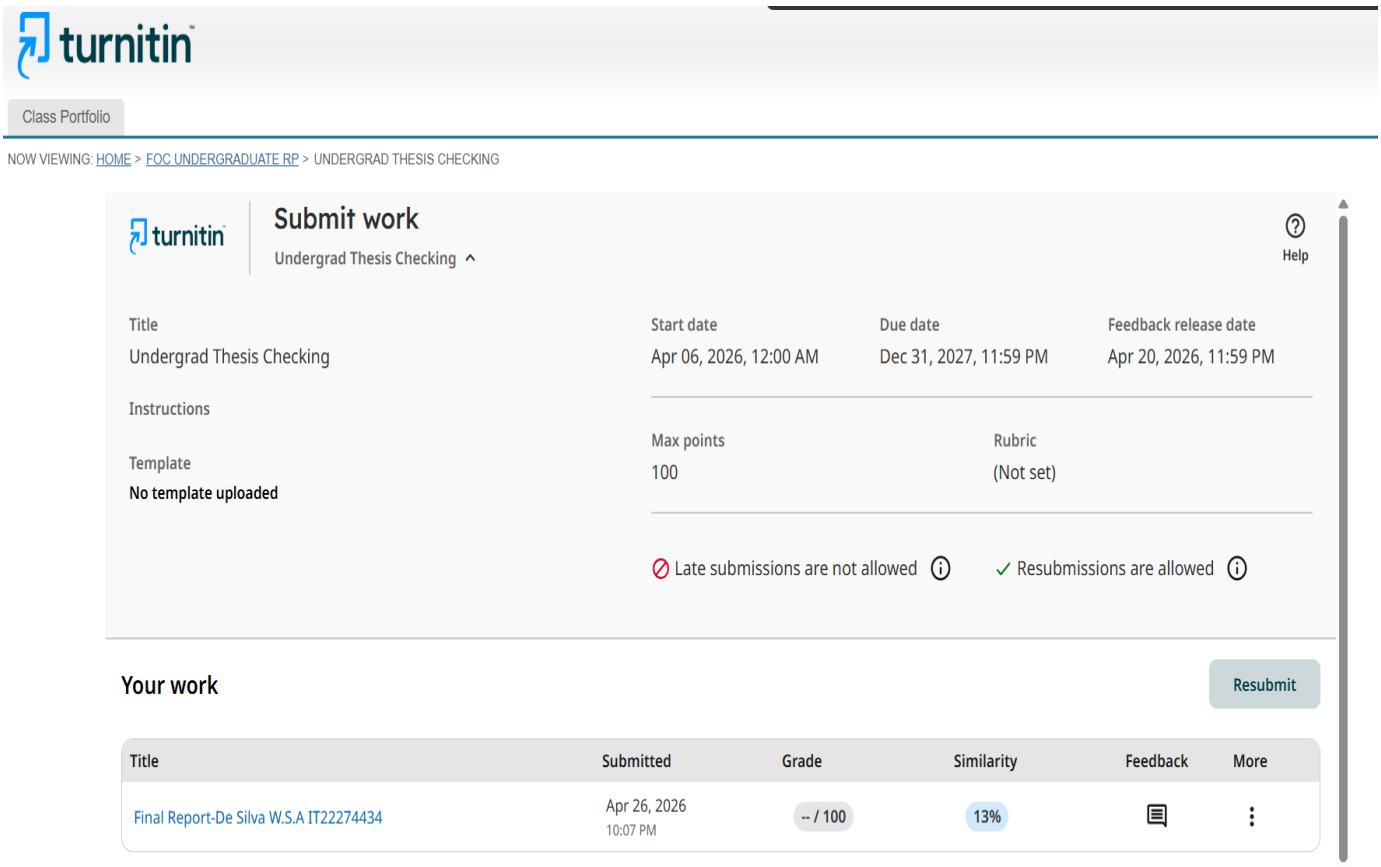
[13] I. &. Loshchilov, "Decoupled weight decay regularization," 2019. [Online].

[14] Y. LeCun, "Deep learning Nature,521(7553), 436-444," 2015. [Online]. Available: https://doi.org/10.1038/nature14539.

[15] J. Barbedo, "A review on the main challenges in automatic plant disease identification based on visible range images," 2016. [Online]. Available: https://doi.org/10.1016/j.biosystemseng.2016.01.017.

[16] M. B. Islam, "Detection of potato diseases using image segmentation and multiclass support vector machine," 2017. [Online].

# 6.Appendice


turnitin
Class Portfolio
NOW VIEWING: HOME > FOC UNDERGRADUATE RP > UNDERGRAD THESIS CHECKING
turnitin
Submit work
Undergrad Thesis Checking
Help
Title
Undergrad Thesis Checking
Instructions
Template
No template uploaded
Start date
Apr 06, 2026, 12:00 AM
Due date
Dec 31, 2027, 11:59 PM
Feedback release date
Apr 20, 2026, 11:59 PM
Max points
100
Rubric
(Not set)
Late submissions are not allowed
Resubmissions are allowed
Your work
Resubmit
Title
Submitted
Grade
Similarity
Feedback
More
Final Report-De Silva W.S.A IT22274434
Apr 26, 2026
10:07 PM
-- / 100
13%